Starter PRD: Session Seat Desk

Completion Theater Hackathon, Agile Testing Days 2026

Build a small application that reserves seats in two fictional workshops. Your team has 60 minutes to build, followed by verification and an independent cross-audit. Any language, framework or AI coding agent is welcome. The product must run on a participant laptop without a paid deployment, a private service or a judge's AI account.

The user and the problem

An event helper needs to see remaining seats, reserve a place for an attendee and cancel a reservation. A successful-looking screen is not enough: the stored state must agree with the response, survive a restart and never oversell a session.

Use fictional aliases only. There is no login, payment, email delivery, public hosting or real conference registration in this exercise. Keep your app and its test data in a fresh, disposable team workspace.

The smallest complete product

Provide a basic user interface and a local JSON HTTP API. The interface may be a simple HTML page. The API lets another team independently inspect behavior. Store state durably, for example in SQLite or a local file. Document the state location. An in-memory array or browser-only state does not meet the persistence requirement. A local process must accept a configurable port and isolated fixture state path, either directly or through a documented adapter. The optional Python scaffold supports PORT and DATA_DIR. Labels, keyboard access and visible error messages belong in the basic UI, not optional creativity work.

Seed exactly these sessions on an explicit fresh reset. Restart must not reset them.

Session ID	Title	Capacity	Initial active reservations
CT-101	Catching Ghost Databases	2	0
CT-102	Tests That Can Fail	3	0

A reservation contains a generated stable ID, session ID, attendee alias, caller-supplied request key and status (active or cancelled). Remaining seats equal capacity minus active reservations. Reject blank or whitespace-only aliases. Trim aliases before comparing repeat requests; comparisons are otherwise case-sensitive. The request key must be a non-empty string.

Required acceptance criteria

Each criterion is worth 5 of the 40 working-app points. PASS requires observed execution against the frozen submission. FAIL or NOT RUN earns 0 for that criterion. Partial implementations can still earn other criteria.

ID	Requirement and observable acceptance
AC-01	Clean start and UI. Another team follows the README, starts the app from the submitted source, opens the UI and sees the two seeded sessions. App startup does not require the builder's account or a hidden service.
AC-02	Session readback. GET /sessions returns both IDs, capacities, active counts and remaining seats. On fresh reset, counts are 0 and remaining seats are 2 and 3. The UI agrees with this response.
AC-03	Reservation. A valid request for CT-101 with alias Ada and key K1 returns success with a stable reservation ID and active status. GET /reservations/{id} returns the same record. Remaining CT-101 seats become 1 and the UI reflects that state.

ID	Requirement and observable acceptance
AC-04	Capacity. After two active CT-101 reservations, a third distinct request returns a non-success response with a clear capacity error and creates no reservation. A fresh one-seat state with simultaneous distinct reservation attempts must produce exactly one success and an active count of 2; all other attempts must reject without state changes. The shared helper repeats ten bounded batches of ten attempts. This probes races; it does not prove absence of all concurrency defects.
AC-05	Idempotency. Repeating the same key, same session and same trimmed alias returns the original reservation ID and current status without consuming another seat. Reusing that key with a different session or alias returns a conflict with no state change. Request keys are globally unique, not just unique within a session.
AC-06	Cancellation. Cancelling an active reservation releases exactly one seat. Repeating cancellation reports cancelled status and leaves the count unchanged. Replaying its original reservation request after cancellation returns that same cancelled reservation; it must not reactivate it. A new request key is needed for a new reservation.
AC-07	Durable state. After creating one active and one cancelled reservation, stop the process, start it again without reset, and use fresh HTTP requests plus the UI to read the same IDs, statuses and seat counts. Refreshing a browser alone is not a restart.
AC-08	Invalid input. Unknown session, blank alias, missing request key, malformed JSON and cancelling an unknown reservation produce non-success responses with clear errors. None may change reservation counts or remaining seats. The process stays available afterward.

Shared API contract

Rules version 1.1, updated after the 17 October dry run. Paths are fixed so another team can test quickly. Use the shared JSON examples in the public repository API contract, or supply a tested adapter for your response field names before the build ends. The optional verifier accepts both a bare session array and the documented wrapper through that adapter. Return valid JSON and appropriate HTTP status codes; any documented 2xx success and 4xx rejection is acceptable. A 200 response containing only an error message is not a rejection.

Method and path	Input	Minimum result
GET /sessions	none	session IDs, titles, capacities, active counts, remaining seats
POST /reservations	sessionId, attendeeAlias, requestKey	stable reservation ID, session, alias, key, status
GET /reservations/{id}	ID from a successful response	stored reservation or 4xx if unknown
POST /reservations/{id}/cancel	empty JSON object	current cancelled record or 4xx if unknown

Example request: {"sessionId":"CT-101","attendeeAlias":"Ada","requestKey":"K1"}.

Provide a documented local reset command for fixture state. It must target only this disposable workspace. Never reset automatically at startup. A reset HTTP endpoint is optional; do not expose one publicly. Auditors can stop and reset their own copy between independent scenarios, but never between the stop and restart steps of AC-07.

Invariants to investigate

- Active count never exceeds capacity and remaining seats never go below 0.

- Replays, rejected input and repeated cancellation do not create extra state changes.

- A successful response agrees with independent stored-state readback.

- Restart preserves records and counts.

- Test failure reaches the process exit status; skipped or absent tests are visible.

Scope and optional ideas

Finish the eight criteria before adding waitlists, search, accessibility improvements or evidence views. Authentication, real personal data, payments, live messaging and production deployment are outside this challenge. Do not invent AI features for the running app: AI is your build assistant, not a required product dependency.

What to hand over

At 21:55 freeze source and tests. Create a source snapshot with a SHA-256 checksum, or use a clean commit with a recorded full hash. Include README run/reset/test commands, runtime version, dependency lockfiles where supported, state location, API field mapping, test source, known gaps and the evidence manifest template. No push or public repository is required. Provide no credentials, AI tokens, private .env files, unrelated history or real attendee records. Use .env.example for configuration names only.

Keep evidence and audit reports outside the frozen source archive. Bind each evidence record to its source checksum or commit. If your source changes before freeze, rerun affected checks. Work after freeze belongs in a separately named copy and does not replace the judged baseline.